
TD Correction (Récursivité) - Correction

Question 1 : Considérons la fonction récursive suivante calculant la somme des éléments d'une liste d'entiers.

```
1 def somme_tab_rec(t, n):
2     # calcule la somme des n premiers éléments de t
3     # Pre-condition : 0 <= n <= len(t)
4     if n == 0:
5         return 0
6     else:
7         return t[n-1] + somme_tab_rec(t, n-1)
```

1. Démontrer que cet algorithme termine.
2. Par récurrence, prouver la correction partielle puis totale de cet algorithme.

Correction Q1.

1. Terminaison :

On choisit comme variant de la récursion n . Initialement $n \geq 0$.

À chaque appel récursif, on effectue un appel sur $n - 1$.

Comme $n > 0$, $n - 1 \geq 0$, ainsi l'algorithme termine sur tout n respectant la pré-condition.

2. Correction :

Soit $P(n)$: L'algorithme retourne $\sum_{i=0}^{n-1} t[i]$

- **Cas de base** ($n = 0$) :

La fonction retourne 0. D'autre part $\sum_{i=0}^{-1} t[i] = 0$. $P(0)$ est vraie.

- **Hérédité** :

Supposons $P(n-1)$ vraie pour $n > 0$. La fonction retourne $\sum_{i=0}^{n-2} t[i]$ pour l'appel `somme_tab_rec(t, n-1)`.

En rentrant dans la fonction pour la valeur n , puisque $n > 0$, la condition `n == 0` est fausse, on va dans le bloc `else`. On retourne ainsi `t[n-1] + somme_tab_rec(t, n-1)`.

En utilisant l'hypothèse de récurrence, l'appel interne a renvoyé $\sum_{i=0}^{n-2} t[i]$. La valeur de retour de cet appel est donc $t[n-1] + \sum_{i=0}^{n-2} t[i] = \sum_{i=0}^{n-1} t[i]$.

- **Conclusion** :

La propriété est initialisée et héréditaire sur $0 \leq n \leq |t|$ donc par récurrence, $P(n)$ est vraie pour tout $n \in [0 : |t|]$. La fonction termine et retourne la somme des n premiers éléments de t .

Question 2 : Considérons l'algorithme d'exponentiation récursive :

```
1 def expo_rec(x, n):
2     # Pre-condition : n >= 0
3     if n == 0:
4         return 1
5     else:
6         return x * expo_rec(x, n - 1)
```

1. Démontrer que cet algorithme termine.
2. Par récurrence, prouvez la correction totale de cet algorithme.

Correction Q2.

1. Terminaison :

On choisit comme variant n . À chaque appel, n devient $n - 1$ et $n \geq 0$. L'algorithme se termine obligatoirement.

2. Correction :

On pose la propriété de récurrence $P(n) : \text{expo_rec}(x, n) = x^n$.

- **Cas de base :**

Si $n = 0$, la fonction retourne 1. Or $x^0 = 1$. Donc $P(0)$ est vraie.

- **Hérédité :**

Supposons $P(n - 1)$ vraie pour $n > 0$, donc $\text{expo_rec}(x, n-1) = x^{n-1}$.

Dans ce cas, l'appel évalue la branche `else`, et retourne la valeur $x \times \text{expo_rec}(x, n-1)$.

En utilisant l'hypothèse de récurrence, on obtient $x \times x^{n-1} = x^n$. $P(n)$ est donc vraie.

- **Conclusion :**

La propriété est initialisée et héréditaire sur $\mathbb{N}$, donc $\text{expo_rec}(x, n) = x^n$ pour toute valeur de $n \in \mathbb{N}$. On en déduit la correction totale de l'algorithme.

Question 3 : Considérons la recherche dichotomique récursive sur une liste `liste` trié dans l'ordre croissant :

```
1 def dico_rec(liste, x, deb, fin):
2     # Pre-condition : deb <= fin, liste trié
3     if deb > fin:
4         return False
5     m = (deb + fin) // 2
6     if liste[m] == x:
7         return True
8     elif liste[m] > x:
9         return dico_rec(liste, x, deb, m - 1)
10    else:
11        return dico_rec(liste, x, m + 1, fin)
```

1. Démontrer que cet algorithme termine en trouvant le variant correspondant.

2. Soit $P(k)$ la propriété « L'algorithme retourne True si et seulement si x est présent entre les indices deb et fin inclusivement, pour une zone de recherche de taille $k = fin - deb + 1$ ».

Prouvez $P(k)$ par récurrence forte sur k . En déduire la correction totale.

Correction Q3.

1. **Terminaison** : Considérons le variant $\text{fin} - \text{deb} + 1$

Si la condition $\text{deb} > \text{fin}$, la fonction termine.

Sinon, on calcule le milieu de cet espace, $m = \lfloor (\text{deb} + \text{fin}) / 2 \rfloor$. Donc, $\text{deb} \leq m \leq \text{fin}$.

Il y a deux appels récursifs possibles :

- $\text{dicho_rec}(t, e, \text{deb}, m - 1)$.
Comme $m \leq \text{fin}$, donc $m - 1 < \text{fin}$ et $m - 1 - \text{deb} + 1 < \text{fin} - \text{deb} + 1$.
- $\text{dicho_rec}(t, e, m + 1, \text{fin})$.
Comme $m \geq \text{deb}$, donc $m + 1 > \text{deb}$ et $\text{fin} - (m + 1) + 1 < \text{fin} - \text{deb} + 1$.

Dans tout les cas le variant diminue strictement, donc l'algorithme termine.

2. **Correction** :

On démontre $P(k)$ par récurrence sur la taille de recherche $k = \text{fin} - \text{deb} + 1$.

- **Cas de base** : Si $k = 0$, donc $\text{deb} > \text{fin}$, la fonction répond **False**, ce qui est vrai car il n'y a pas d'élément. $P(0)$ est vrai.
- **Hérédité** : Soit $k > 0$ et admettons la propriété forte $P(k')$ vraie pour tout $k' < k$.

On distingue 3 cas :

- Si $\text{liste}[m] == x$, on retourne **True**, ce qui est vrai car l'élément est trouvé.
- Si $\text{liste}[m] > x$, alors puisque la liste est croissante, si l'élément apparaît de ce segment d'index entre deb et fin , il ne peut l'être qu'avant m . Or on appelle récursivement $\text{dicho_rec}(t, e, \text{deb}, m - 1)$. Comme k diminue strictement à chaque appel, on peut appliquer l'hypothèse de récurrence.
- La réciproque à droite est logiquement la copie de la précédente.

On en conclue que dans tout les cas $P(k)$ est vraie par l'hypothèse de récurrence.

- **Conclusion** : L'algorithme termine et est partiellement correct, il est donc totalement correct.